

Cache Simulator Source Code (cache_sim.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

unsigned long long seen_hash[1000003];

int check_and_add(unsigned long long block_addr) {
    unsigned long long val = block_addr + 1;
    int idx = (int)(block_addr % 1000003);
    while (seen_hash[idx] != 0) {
        if (seen_hash[idx] == val) {
            return 1;
        }
        idx = (idx + 1) % 1000003;
    }
    seen_hash[idx] = val;
    return 0;
}

int main(int argc, char **argv) {
    if (argc < 3) {
        printf("Usage: ./cache_sim config.txt trace.txt\n");
        return 1;
    }

    FILE *cfg = fopen(argv[1], "r");
    if (cfg == NULL) {
        return 1;
    }

    int cache_size = 0;
    int block_size = 0;
    int associativity = 0;
    char replacement_policy[100] = "";
    char write_policy[100] = "";
    char write_allocate[100] = "";
    int address_bits = 0;
    double hit_time = 0.0;
    char hit_unit[50] = "";
```

```

double miss_time = 0.0;
char miss_unit[50] = "";

char line[256];
while (fgets(line, sizeof(line), cfg)) {
    char *hash = strchr(line, '#');
    if (hash != NULL) {
        *hash = '\0';
    }
    char key[100] = "";
    char val[100] = "";
    if (sscanf(line, " %[^\n] = %[^\n]", key, val) == 2) {
        int k_len = strlen(key);
        while (k_len > 0 && (key[k_len - 1] == ' ' || key[k_len - 1] ==
'\t' || key[k_len - 1] == '\n' || key[k_len - 1] == '\r')) {
            key[k_len - 1] = '\0';
            k_len--;
        }
        int k_start = 0;
        while (key[k_start] == ' ' || key[k_start] == '\t') {
            k_start++;
        }
        char clean_key[100];
        strcpy(clean_key, key + k_start);

        int v_len = strlen(val);
        while (v_len > 0 && (val[v_len - 1] == ' ' || val[v_len - 1] ==
'\t' || val[v_len - 1] == '\n' || val[v_len - 1] == '\r')) {
            val[v_len - 1] = '\0';
            v_len--;
        }
        int v_start = 0;
        while (val[v_start] == ' ' || val[v_start] == '\t') {
            v_start++;
        }
        char clean_val[100];
        strcpy(clean_val, val + v_start);

        if (strcmp(clean_key, "cache_size") == 0) {
            cache_size = atoi(clean_val);
        } else if (strcmp(clean_key, "block_size") == 0) {
            block_size = atoi(clean_val);
        } else if (strcmp(clean_key, "associativity") == 0) {
            associativity = atoi(clean_val);
        }
    }
}

```

```

        } else if (strcmp(clean_key, "replacement_policy") == 0) {
            strcpy(replacement_policy, clean_val);
        } else if (strcmp(clean_key, "write_policy") == 0) {
            strcpy(write_policy, clean_val);
        } else if (strcmp(clean_key, "write_allocate") == 0) {
            strcpy(write_allocate, clean_val);
        } else if (strcmp(clean_key, "address_bits") == 0) {
            address_bits = atoi(clean_val);
        } else if (strcmp(clean_key, "hit_time") == 0) {
            sscanf(clean_val, "%lf %s", &hit_time, hit_unit);
        } else if (strcmp(clean_key, "miss_time") == 0) {
            sscanf(clean_val, "%lf %s", &miss_time, miss_unit);
        }
    }
}

fclose(cfg);

int num_blocks = cache_size / block_size;
int num_sets = num_blocks / associativity;

unsigned long long *main_tag = (unsigned long long*)calloc(num_blocks,
sizeof(unsigned long long));
int *main_valid = (int*)calloc(num_blocks, sizeof(int));
int *main_dirty = (int*)calloc(num_blocks, sizeof(int));
unsigned long long *main_count = (unsigned long long*)calloc(num_blocks,
sizeof(unsigned long long));

unsigned long long *fa_tag = (unsigned long long*)calloc(num_blocks,
sizeof(unsigned long long));
int *fa_valid = (int*)calloc(num_blocks, sizeof(int));
int *fa_dirty = (int*)calloc(num_blocks, sizeof(int));
unsigned long long *fa_count = (unsigned long long*)calloc(num_blocks,
sizeof(unsigned long long));

FILE *tr = fopen(argv[2], "r");
if (tr == NULL) {
    free(main_tag);
    free(main_valid);
    free(main_dirty);
    free(main_count);
    free(fa_tag);
    free(fa_valid);
    free(fa_dirty);
    free(fa_count);
}

```

```

        return 1;
    }

    long long total = 0;
    long long reads = 0;
    long long writes = 0;
    long long hits = 0;
    long long misses = 0;
    int dirty_evict = 0;

    long long compulsory = 0;
    long long conflict = 0;
    long long capacity = 0;

    unsigned long long access_counter = 0;

    char tr_line[256];
    while (fgets(tr_line, sizeof(tr_line), tr)) {
        char op_str[10] = "";
        char addr_str[100] = "";
        if (sscanf(tr_line, "%s %s", op_str, addr_str) != 2) {
            continue;
        }
        char op = op_str[0];
        if (op >= 'a' && op <= 'z') {
            op = op - 'a' + 'A';
        }
        if (op != 'R' && op != 'W') {
            continue;
        }

        unsigned long long address = strtoull(addr_str, NULL, 16);
        total++;
        if (op == 'R') {
            reads++;
        } else {
            writes++;
        }

        unsigned long long block_addr = address / block_size;

        int set_idx = block_addr % num_sets;
        unsigned long long tag = block_addr / num_sets;
        int start = set_idx * associativity;

```

```

int hit_way = -1;
for (int i = 0; i < associativity; i++) {
    if (main_valid[start + i] && main_tag[start + i] == tag) {
        hit_way = i;
        break;
    }
}

access_counter++;
int main_hit = 0;
if (hit_way != -1) {
    main_hit = 1;
    if (op == 'W' && strcmp(write_policy, "write_back") == 0) {
        main_dirty[start + hit_way] = 1;
    }
    if (strcmp(replacement_policy, "LRU") == 0) {
        main_count[start + hit_way] = access_counter;
    }
} else {
    if (op == 'R' || strcmp(write_allocate, "yes") == 0) {
        int target = -1;
        for (int i = 0; i < associativity; i++) {
            if (!main_valid[start + i]) {
                target = i;
                break;
            }
        }
        if (target == -1) {
            if (strcmp(replacement_policy, "Random") == 0) {
                target = rand() % associativity;
            } else {
                unsigned long long min_val = main_count[start];
                target = 0;
                for (int i = 1; i < associativity; i++) {
                    if (main_count[start + i] < min_val) {
                        min_val = main_count[start + i];
                        target = i;
                    }
                }
            }
        }
        if (main_dirty[start + target]) {
            dirty_evict++;
        }
    }
}

```

```

        main_valid[start + target] = 1;
        main_tag[start + target] = tag;
        main_dirty[start + target] = (op == 'W' &&
strcasecmp(write_policy, "write_back") == 0) ? 1 : 0;
        main_count[start + target] = access_counter;
    }
}

int fa_hit_way = -1;
for (int i = 0; i < num_blocks; i++) {
    if (fa_valid[i] && fa_tag[i] == block_addr) {
        fa_hit_way = i;
        break;
    }
}

int fa_hit = 0;
if (fa_hit_way != -1) {
    fa_hit = 1;
    if (op == 'W' && strcasecmp(write_policy, "write_back") == 0) {
        fa_dirty[fa_hit_way] = 1;
    }
    if (strcasecmp(replacement_policy, "LRU") == 0) {
        fa_count[fa_hit_way] = access_counter;
    }
} else {
    if (op == 'R' || strcasecmp(write_allocate, "yes") == 0) {
        int target = -1;
        for (int i = 0; i < num_blocks; i++) {
            if (!fa_valid[i]) {
                target = i;
                break;
            }
        }
    }
    if (target == -1) {
        if (strcasecmp(replacement_policy, "Random") == 0) {
            target = rand() % num_blocks;
        } else {
            unsigned long long min_val = fa_count[0];
            target = 0;
            for (int i = 1; i < num_blocks; i++) {
                if (fa_count[i] < min_val) {
                    min_val = fa_count[i];
                    target = i;
                }
            }
        }
    }
}

```

```

        }
    }
}

fa_valid[target] = 1;
fa_tag[target] = block_addr;
fa_dirty[target] = (op == 'W' && strcasecmp(write_policy,
"write_back") == 0) ? 1 : 0;
fa_count[target] = access_counter;
}
}

if (main_hit) {
    hits++;
} else {
    misses++;
    int seen = check_and_add(block_addr);
    if (seen == 0) {
        compulsory++;
    } else {
        if (fa_hit) {
            conflict++;
        } else {
            capacity++;
        }
    }
}
}

fclose(tr);

double hit_rate = 0.0;
double miss_rate = 0.0;
double amat = hit_time;
if (total > 0) {
    hit_rate = ((double)hits / total) * 100.0;
    miss_rate = ((double)misses / total) * 100.0;
    amat = hit_time + ((double)misses / total) * miss_time;
}

FILE *out = fopen("output.txt", "w");
if (out == NULL) {
    free(main_tag);
    free(main_valid);
    free(main_dirty);
}

```

```

        free(main_count);
        free(fa_tag);
        free(fa_valid);
        free(fa_dirty);
        free(fa_count);
        return 1;
    }

    char sz_str[50] = "";
    if (cache_size >= 1024 * 1024) {
        sprintf(sz_str, "%d MB", cache_size / (1024 * 1024));
    } else if (cache_size >= 1024) {
        sprintf(sz_str, "%d KB", cache_size / 1024);
    } else {
        sprintf(sz_str, "%d B", cache_size);
    }

    char blk_str[50] = "";
    if (block_size >= 1024 * 1024) {
        sprintf(blk_str, "%d MB", block_size / (1024 * 1024));
    } else if (block_size >= 1024) {
        sprintf(blk_str, "%d KB", block_size / 1024);
    } else {
        sprintf(blk_str, "%d B", block_size);
    }

    fprintf(out, "Cache Configuration:\n");
    fprintf(out, "Cache Size: %s\n", sz_str);
    fprintf(out, "Block Size: %s\n", blk_str);
    fprintf(out, "Associativity: %d-way\n", associativity);
    fprintf(out, "Replacement: %s\n", replacement_policy);
    fprintf(out, "Write Policy: %s\n", write_policy);
    fprintf(out, "Write Allocate: %s\n\n", write_allocate);

    fprintf(out, "Results:\n");
    fprintf(out, "Total Accesses: %lld\n", total);
    fprintf(out, "Reads: %lld\n", reads);
    fprintf(out, "Writes: %lld\n\n", writes);

    fprintf(out, "Hits: %lld\n", hits);
    fprintf(out, "Misses: %lld\n", misses);
    fprintf(out, "Dirty Evictions: %d\n\n", dirty_evict);

    fprintf(out, "Compulsory Misses: %lld\n", compulsory);

```



```
fprintf(out, "Conflict Misses: %lld\n", conflict);
fprintf(out, "Capacity Misses: %lld\n\n", capacity);

fprintf(out, "Hit Rate: %.2f%%\n", hit_rate);
fprintf(out, "Miss Rate: %.2f%%\n\n", miss_rate);

fprintf(out, "AMAT: %.2f %s\n", amat, hit_unit);
fclose(out);

free(main_tag);
free(main_valid);
free(main_dirty);
free(main_count);
free(fa_tag);
free(fa_valid);
free(fa_dirty);
free(fa_count);

return 0;
}
```